MASTER PROMPT — Build "FORGEBOUND" (working title), a complete offline anime-style blacksmith-RPG for Android

> Paste this whole document into your coding agent as ONE prompt. It is self-contained. Do not ask me questions. Decide, build, verify, deliver.

---

0. YOUR ROLE AND ABSOLUTE RULES

You are a senior game developer, technical artist, game designer, and QA engineer in one. You will build a **complete, polished, fully offline Android game** from an empty folder to a signed, installable APK/AAB, in a single autonomous run.

Non-negotiable rules:

1. **No half-built game.** No stubs, no TODO, no "coming soon" buttons, no placeholder art, no lorem ipsum, no dead menu items, no mocked screens. Every screen, button, system, item, quest and upgrade listed here must work end-to-end.
2. **Do not work stage-by-stage asking for approval.** Plan once, then execute the whole plan in one run. Do not stop to ask questions; when something is ambiguous, pick the best option, write the decision into `docs/DECISIONS.md`, and continue.
3. **100% offline at runtime.** The shipped app must declare **no INTERNET permission** and make **no network calls**. All assets are bundled. (You may use the internet at *build time* only, see Section 12.)
4. **Use every capability you have**: internet access to download tools/libraries/free assets, image generation, audio generation/synthesis, code execution, file system, emulator/Gradle builds, screenshot capture, automated tests. Use them to the maximum.
5. **Original IP.** The game is *inspired by* the "inherit a legendary blacksmith's power in a VR game and become overpowered by crafting" genre. Do **not** copy any existing franchise's characters, names, logos, art, music or text. Everything (names, story, art, music) must be original.

6. **Self-verify.** Build it, install it on an emulator, play it with scripted input, capture screenshots, fix every crash/bug you find. Do not declare done until the Definition of Done (Section 16) is fully met.
7. If a context/token limit interrupts you, keep a living `docs/PROGRESS.md` checklist so you can resume exactly where you stopped, and continue until everything is complete.

---

1. GAME CONCEPT

Title: Forgebound (Persian display name: «آهنگرِ برگزیده»)

Genre: Anime-style action-idle RPG with a deep crafting/forging core. Portrait or landscape (choose landscape, lock it, with safe-area handling).

Fantasy: The player is *Kai*, an unlucky young man who is dragged into the virtual-reality world **"Aetherion"** and inherits the lost legacy of the legendary smith **Master Vulcan**. Kai becomes an overpowered hero not by grinding monsters alone but by **forging impossible gear**, outcrafting the world.

Pillars:

- Forging is the star: a satisfying, skill-based, juicy forging minigame.
- Gear is power: items define the build; stats, sets and enchants matter.
- Constant visible progression: always something to upgrade, unlock, or craft.
- Anime presentation: bold key-art, cut-in portraits, dramatic skill effects, expressive dialogue.
- Session-friendly: great in 3-minute sessions and in long sessions; strong offline-progress system.

Story (original, ~10 chapters, fully written): Kai wakes in Aetherion, a fantasy world whose items are weak and brittle. He finds Vulcan's Anvil, learns the Seven Sacred Techniques, gathers allies (a swift rogue, a shy mage, a veteran knight, a merchant), and faces the Rust King who is corrupting all metal in the world. Write the full script: intro, 10 chapters, each with 3-6 dialogue scenes, 1 boss, and an ending + epilogue + post-game hint. All text must be written and localized (Section 11).

---

2. TECH STACK (decided, do not debate)

- **Language:** Kotlin. **UI/Rendering:** Jetpack Compose for menus + a custom `Canvas` / `SurfaceView`-style 2D game loop (or LibGDX if you judge it clearly better for effects/performance — decide once, document it).
- **Min SDK 24, Target SDK latest stable, 64-bit, arm64 + x86_64.**
- **Persistence:** Room (SQLite) + a versioned save format, with autosave, backup file export/import (local, user-picked file), and corruption recovery (keep last 3 saves).
- **Audio:** SoundPool/ExoPlayer or AudioTrack; looped BGM, SFX pool, ducking.
- **DI:** simple manual DI or Hilt. **Architecture:** MVVM + unidirectional data flow, game logic in pure Kotlin modules that are unit-testable with no Android dependency.
- **Build:** Gradle (Kotlin DSL), R8 minify + shrink resources, ABI splits optional. Final APK/AAB size target ⩽ 150 MB.
- **Permissions:** only vibration (optional). **No INTERNET, no ads SDK, no analytics, no trackers.**
- **Accessibility:** scalable text, color-blind-safe rarity (colors + icons + border shapes), haptics toggle, reduce-motion toggle.

---

3. ARCHITECTURE & PROJECT LAYOUT

Create a clean multi-module or package-per-feature project:

- `core/` — math, RNG (seeded, deterministic for tests), time, event bus, balance constants.
- `data/` — Room entities/DAOs, save/load, migration, content loaders (JSON content packs in `assets/content/`).
- `game/` — combat, forging, crafting, stats, loot, quests, economy, offline progress, achievements.
- `ui/` — Compose screens, theme, components, navigation.
- `render/` — sprite batching, particles, camera shake, screen effects, shaders (AGSL/GLSL where available, with fallback).
- `audio/`, `i18n/`, `tools/` (asset pipeline scripts), `docs/`, `tests/`.

All content (items, enemies, quests, dialogue, shop, balance) lives in **data files (JSON)**, not hard-coded, so balance can be tuned without code changes. Provide a **content validator script** that fails the build on broken references.

---

4. CORE LOOP

1. **Explore / Fight** in a zone (auto-battle with player-triggered skills) → earn **Gold, Materials, XP, Drops**.
2. **Forge** weapons/armor/accessories from materials via the forging minigame → quality grade determines stats.
3. **Upgrade** (enhance, reforge, enchant, socket, awaken) gear; **level up** and spend stat/skill points.
4. **Shop / Market** to buy blueprints, materials, consumables; sell surplus.
5. **Quests / Chapters / Achievements** drive goals and rewards.
6. Push **bosses, tower, and dungeons** to unlock new zones and higher-tier materials. Repeat with ever-bigger power spikes.

---

5. SYSTEMS (implement ALL fully)

5.1 Player stats & leveling

Base stats: **STR, DEX, VIT, INT, LUCK**. Derived: HP, MP, ATK, MAG, DEF, RES, CRIT %, CRIT DMG, SPEED, EVASION, DROP RATE, FORGE SKILL.

- Max level 120. XP curve: `xp(L) = round(80 * L^1.65)`. Each level: +5 stat points and +1 skill point every 2 levels.
- **DEX drives forging**: affects forging success window, quality ceiling and crafting speed (signature mechanic).
- Stat respec: unlimited, costs gold scaling with level.

5.2 Classes / Hidden Legacy

Start as **Novice → at Lv10 choose a path**: Knight, Rogue, Mage, or Ranger. Plus the **hidden legacy class "Vulcan's Heir"** unlocked by completing a secret quest chain (Section 8); it adds the unique **Forge Mastery tree**: forging bonuses, self-buffing crafted gear, and the **"Overgear" passive** (equipped item effects scale beyond normal caps). Each class has its own skill tree (Section 5.5).

5.3 Combat (real-time, auto-battle + manual skills)

- Side-view 2D lane combat vs waves. Player + up to 2 AI allies.

- Auto attack by SPEED; **tap skill buttons** (4 active slots + 1 ultimate with a gauge). **Dodge/guard** button with perfect-timing windows.
- Damage formula: `dmg = (ATK * skillMult) * (100 / (100 + enemyDEF)) * critMult * elementMult * rand(0.95,1.05)`. Elements: Fire, Ice, Lightning, Earth, Light, Dark, with a weakness wheel.
- Status effects: burn, freeze, shock, poison, bleed, stun, rust (special), buffs/debuffs with stacking rules.
- Combo counter, hit-stop, screen shake, damage numbers, crit flashes.
- Enemies: **at least 40 normal enemies, 12 elites, 10 chapter bosses, 3 raid bosses, 1 final boss**, each with unique patterns, telegraphed attacks, and phases.
- Death: respawn at zone entrance with a small gold penalty; never lose items.

5.3a Zones & Dungeons

- **10 story zones** (e.g., Whispering Plains, Emberdeep Mines, Frostveil Peaks, Sunken Library, Stormglass Desert, Gloamwood, Skyforge Ruins, Crimson Citadel, Void Rift, Rust King's Throne). Each has 10 stages + a boss + mini-events.
- **Daily dungeons** (material farms, rotating by weekday using device date, anti-clock-cheat check).
- **Endless Tower** (100+ floors, scaling, weekly-style milestone rewards using in-game day counter).
- **Raid bosses** with unique mechanics and rare-material drops.

5.4 Forging system (the signature feature)

Blueprints define recipe: required materials, base stats, tier, slot. Forging is an **interactive multi-phase minigame** (≈10-20 s), all phases skippable by an "Auto-Forge" that yields lower average quality:

1. **Heat** — hold to heat the ingot into the green zone; overheating risks flaws.
2. **Hammer** — rhythm hits: tap on beats; Perfect/Great/Good/Miss feed a **Quality Meter**. DEX widens timing windows.
3. **Shape** — drag along the guide curve on the blade/armor outline (accuracy %).
4. **Quench** — a timed pull/release for hardness.
5. **Polish** — quick swipe fill for the final shine bonus.

Result: **Quality score 0-100 → Grade**: Crude, Common, Fine, Superior, Masterwork, Flawless, **Legendary** (needs Perfect streak + high FORGE SKILL + rare materials). Grade multiplies base stats (×0.8 … ×2.2 + chance of a bonus affix). **"Divine Hammering"**: rare proc that rolls a hidden bonus stat and a unique visual aura.

- **Item durability** (optional soft system): repairs cost gold, never destroy items.
- **Salvage** items to get materials back; **Reforge** re-rolls affixes with a lock option.
- **Enhancement +0 → +20** with success/failure rates, pity counter, and "protect" scrolls (failure never deletes items; at worst drops the level by 1).
- **Enchanting** (rune slots), **Socketing** gems, **Awakening** (tier up), **Set bonuses** (2/4/6-piece), **Transmog** (cosmetic skin of any owned item).

5.5 Skills

- Per class: **24 active + passive skills**, plus 8 shared skills and the Forge Mastery tree (12 nodes). Skills level 1-10 with clear scaling and cooldown/MP cost data. Each skill has a custom animation + VFX + SFX (Section 9-10).
- Ultimates: one per class, with a full-screen anime cut-in.

5.6 Items & rarity

Slots: Weapon (sword, greatsword, dagger, bow, staff, hammer), Helm, Chest, Gloves, Boots, Cape, Ring ×2, Amulet, Charm. Rarity: Common (grey), Uncommon (green), Rare (blue), Epic (purple), Legendary (gold), **Mythic** (red-prismatic, only via forging). Provide **at least 200 unique forgeable items + 40 drop-only uniques + 12 mythic sets**. Every item has name, flavor text (localized), icon, stats, tier, and sources.

- Inventory: 200 slots expandable via shop, sort/filter/search, lock, bulk sell/salvage with safety confirm, compare tooltip.
- Materials: 60+ types across tiers (ores, woods, leather, gems, essences, rare monster parts).

5.7 Allies / Party

- **5 recruitable allies** (story-based), each with portrait, affinity levels 1-10, skills, a personal side-quest chain, and equipment slots (Kai can forge gear for them — big affinity boost).

5.8 Pets (light system)

- 6 collectible companions with passive buffs, feeding with materials, evolution at levels 10/20.

5.9 Economy & Shop

Currencies: **Gold** (main), **Aether Shards** (rare, earned via quests/tower, **never purchasable with real money — no IAP at all**), **Forge Tokens** (event/dungeon), **Fame** (achievement-based unlock currency).

- **Shops (all functional):** General Store, Blueprint Merchant (rotating stock refreshes every N in-game minutes/day), Material Exchange, Gem Cutter, Rune Master, Cosmetic Tailor (transmogs, UI themes, forge-effect skins), Pet Keeper, Black Market (rare, high price, limited daily). Sinks must exist at every stage to keep the economy balanced.
- Dynamic prices: soft supply/demand (selling many of one item reduces its price temporarily).
- Every price, drop rate and upgrade cost lives in JSON with a **balance spreadsheet** exported to `docs/BALANCE.csv`.

5.10 Quests & tasks

- **Main story**: 10 chapters (≈60 quests).
- **Side quests**: ⩾ 60, including ally chains and the hidden Vulcan chain (7 steps).
- **Daily tasks** (6/day: fight X, forge X, enhance X, spend gold, clear dungeon, use skill) with a daily-streak reward ladder (day counter based on device date).
- **Weekly tasks** (4/week) with a big chest.
- **Achievements**: ⩾ 120 with Fame rewards and titles; a trophy-room screen showing them.
- **Collections**: item codex, monster bestiary, material atlas — completion rewards.
- A **Quest Log** UI with tracking, auto-pathing hints ("go to Zone 3 Stage 4") and claim-all button.

5.11 Offline / idle progress

While the app is closed, the hero keeps auto-farming the last cleared stage at a reduced rate (cap 12 h, upgradeable to 24 h). On open show an **Idle Report** (gold, materials, XP, drops). Anti-cheat: compare device time against a monotonic elapsed-realtime snapshot; reject large backward clock jumps.

5.12 Progression pacing (target)

- First 10 minutes: tutorial, first forge, first upgrade, first boss tease.
- Hour 1: classes, shop, 2 zones. Hours 2-10: zones 3-7, sets, enchants. Hours 10-25: zones 8-10, raids, tower, mythics. Endgame loops: tower, raids, mythic hunting, 100% codex.

5.13 Tutorial

Fully scripted, interactive, skippable, replayable from Settings, with contextual tips on first use of each system and a **Help/Encyclopedia** screen.

---

6. FULL SCREEN LIST (all required and polished)

Splash/Logo → Title (animated key-art, Continue/New Game/Settings) → Save Slots (3) → Main Hub (Smithy view) → Map/Zone Select → Battle → Forging Minigame → Forge/Blueprint Browser → Inventory/Equipment → Item Detail & Compare → Enhance/Reforge/Enchant/Socket/Awaken → Character & Stats → Skill Trees → Allies/Party → Pets → Shops (8 variants) → Quest Log → Achievements/Codex/Bestiary → Daily/Weekly → Tower → Raids → Idle Report → Story/Dialogue (visual-novel style) → Cutscene/Cut-in → Results/Loot screen → Settings (audio, graphics, language, haptics, reduce-motion, text size, backup export/import, credits) → Help/Encyclopedia → Credits → Privacy note (offline, no data collected).

Navigation: bottom bar in hub, back-button handling, no dead ends, state restored after process death.

---

7. ART DIRECTION (anime style, original)

- **Style:** modern high-quality anime — clean line art, cel-shaded colors with soft gradients, saturated key colors, dramatic rim-lighting, painterly backgrounds. Consistent palette per zone.
- **Theme:** warm forge palette (ember orange, molten gold, steel blue) against moody fantasy zones; UI uses brushed-metal, parchment and rune motifs with crisp readable typography.
- **Required art (generate or create ALL; no placeholders):**
  - Key art + title screen; app icon (adaptive, with monochrome layer) + feature graphic.

  - Protagonist Kai in 6+ equipment-tier visual stages, 5 allies, Master Vulcan, Rust King, and all enemies/bosses (sprite sheets or layered/rigged 2D with idle/attack/hit/death animations).
  - Dialogue portraits with ≥ 5 expressions for each main character; boss and ultimate cut-in illustrations.
  - 10 zone backgrounds with parallax layers (3-5 layers each), the Smithy hub (day/night), shop interiors.
  - **Icons for every item, material, skill, status, currency and achievement** (consistent set, rarity frames).
  - UI kit: buttons, panels, bars, tabs, toasts, dialogs, in-game fonts supporting **Persian and Latin** (use open-license fonts).
- **Method:** use the strongest image-generation tools available to you for illustrations, then post-process (background removal, atlas packing, palette consistency, mipmaps/compression to WebP/ASTC). Keep a **style bible** (`docs/STYLE_BIBLE.md`) with prompts/seeds, palettes and rules so all assets match. Reject and regenerate inconsistent outputs.
- Where generation is unreliable for animation, build animations with **skeletal/tween rigs, layered parts, squash-and-stretch, smear frames and procedural motion** rather than shipping static sprites.

---

8. SECRET CONTENT

The hidden "Vulcan's Heir" chain: 7 steps with environmental clues across zones; unlocks the class, a secret zone, the best mythic set, and the true ending. Include 10+ easter eggs and 3 secret bosses.

---

9. EFFECTS, "JUICE" & VFX (go all out, keep 60 FPS)

- **Particle system** (pooled): sparks, embers, smoke, slashes, magic circles, shockwaves, lightning arcs, ice shards, petals, glowing runes.
- **Forging VFX:** heat shimmer, spark bursts per hammer hit scaled by timing grade, quench steam, final "reveal" with light rays and rarity-colored aura; Legendary/Mythic get a full-screen cinematic flourish.

- **Combat VFX:** unique per skill and element, hit-stop, camera shake/zoom, chromatic aberration pulse, motion blur streaks, screen flash, slow-mo on ultimate, anime speed-lines.
- **Shaders** (AGSL on API 33+, with graceful Canvas fallbacks): glow/bloom approximation, dissolve (for item reveal), outline, shimmer, heat distortion, vignette.
- **UI motion:** springy transitions, number count-ups, loot pop with rarity beams, reward chest opening animation, level-up burst.
- **Quality tiers** (Low/Medium/High) auto-detected + manual; a **performance budget**: ≥ 60 FPS on mid-range devices (e.g., 4 GB RAM, Snapdragon 6xx-class), cold start < 3 s, memory < 600 MB.

---

10. AUDIO

- **BGM:** 12+ original tracks (title, hub, 10 zones/boss themes, forging theme, victory, final boss) with seamless loops. **SFX:** 150+ (UI, hammer on metal by grade, quench, swords, magic by element, loot by rarity, level-up, quest complete, etc.). Optional short anime voice-style "shouts" are allowed only if you can generate original ones; otherwise skip.
- Create via procedural synthesis/original generation or **CC0/public-domain** sources only. Keep `docs/CREDITS.md` and `docs/LICENSES.md` with every asset's source and license. Mix to consistent loudness, compress (OGG/Opus).

---

11. LOCALIZATION

- **Persian (fa, full RTL support) and English (en)** — both complete, no missing strings; default = device language, switchable at runtime.
- Persian text must render correctly (shaping, numerals toggle Persian/Latin digits, RTL layout mirroring, correct fonts). Dialogue/lore written natively in both languages by you (not machine-literal). Add a build check that fails on missing keys.

---

12. ASSET & TOOLING PIPELINE (use internet and generation to the max)

- Use the internet at **build time** to: install SDK/Gradle/dependencies, download open-license (CC0/OFL/Apache) fonts, tools, and reference materials; run image/audio generation; build atlases and compress assets; capture emulator screenshots.

- Write reproducible scripts in `tools/` (`gen_assets`, `pack_atlas`, `validate_content`, `build_release`), so the whole project can be rebuilt from scratch with one command.
- Never ship anything without recording its license. No scraping of copyrighted anime/game assets.

---

13. SAVE, SETTINGS & ROBUSTNESS

- Autosave on every major event and on app pause; atomic writes; schema migrations with tests; 3 save slots; export/import backup file.
- Handle process death, rotation lock, low memory, background/foreground, interruptions (calls), notch/cutouts, gesture-nav, tablets/foldables.
- Crash-safe: global exception handler writes a local log; no data collection or upload.
- Cheat resistance (offline-reasonable): checksum on saves, time-tamper checks.

---

14. BALANCE & CONTENT TARGETS (minimums)

- Items ≥ 240 · Materials ≥ 60 · Skills ≥ 110 · Enemies ≥ 65 (incl. bosses) · Zones 10 + secret · Quests ≥ 130 · Achievements ≥ 120 · Dialogue scenes ≥ 80 · Shops 8 · Pets 6 · Allies 5 · BGM ≥ 12 · SFX ≥ 150.
- Provide a **balance simulator** (headless Kotlin/Python) that simulates progression and verifies time-to-clear targets, economy health (no infinite-gold exploits), and drop rates; output `docs/BALANCE_REPORT.md`. Fix any exploit found.

---

15. TESTING & QA (must run and pass)

- Unit tests: damage formula, forging scoring, drop tables, enhancement odds, economy sinks, save/migrate, offline progress, localization completeness.
- Instrumented/UI tests (Espresso/Compose test or scripted `adb`): full new-game flow, first forge, first fight, shop buy/sell, quest claim, save/restore, language switch.
- **Automated playthrough bot** that plays Chapter 1-3 headlessly to prove nothing is soft-locked; plus a soak test (30 min) checking memory leaks and FPS.
- Run on at least 3 emulator profiles (phone small, phone large, tablet). Capture screenshots of every screen into `docs/screenshots/`.

- Static checks: lint, ktlint/detekt, no `TODO` / `FIXME` /placeholder strings (CI-style grep must return zero).

---

16. DELIVERABLES & DEFINITION OF DONE

Deliver in the project root:

1. Full source code (clean, commented where non-obvious) + Gradle project that builds with one command.
2. **Signed release APK** and AAB (generate a keystore, store its details securely in `docs/SIGNING.md`) + a debug APK.
3. `README.md` (how to build/run/test), `docs/GDD.md` (full game design document), `docs/DECISIONS.md` , `docs/STYLE_BIBLE.md` , `docs/BALANCE.csv` , `docs/BALANCE_REPORT.md` , `docs/CREDITS.md` , `docs/LICENSES.md` , `docs/PROGRESS.md` , `docs/TEST_REPORT.md` .
4. Screenshots + a short gameplay capture (GIF/MP4) from the emulator.

The job is DONE only when ALL are true:

- [ ] App installs and runs on emulator with **no INTERNET permission** (verify in the manifest and with `aapt`).
- [ ] Every screen in Section 6 exists, is reachable, and works; no placeholders anywhere.
- [ ] The full story (10 chapters + ending) is playable start to finish; secret chain works.
- [ ] Forging minigame, enhancement, enchant, socket, awaken, reforge, salvage, transmog all work.
- [ ] All 8 shops, daily/weekly tasks, achievements, codex, tower, raids, pets, allies work.
- [ ] Offline idle progress and save/backup/restore verified.
- [ ] Persian + English complete, RTL correct.
- [ ] Content minimums (Section 14) met; validator passes.
- [ ] All tests pass; bot playthrough passes; no crashes in soak test; ≥ 60 FPS target met on the mid-range profile.
- [ ] Asset licenses documented; zero copyrighted third-party assets.

---

17. WORK PROTOCOL (how you execute)

1. Spend your first output creating `docs/PLAN.md` (a complete task breakdown mapped to this prompt) and `docs/PROGRESS.md` (checkbox list). Then **execute everything without waiting for me**.
2. Build in this internal order (without pausing for review): project skeleton & content pipeline → core systems & data → UI shell & navigation → combat → forging → items/economy/shops → quests/story → art & VFX & audio generation/integration → localization → balance simulation → tests → performance pass → polish → release build → final verification against Section 16.
3. After each internal milestone, run the build and tests; fix everything before moving on.
4. At the very end, print a final report: what was built, how to run it, test results, known limitations (should be none that violate the rules above), and the paths of the APK/AAB.

Begin now. Build the complete game. Do not stop until Section 16 is fully satisfied.